Operating System Practicum

Modul 10 - Symbol Export & Interaction

Case Study

CASE STUDY PROVISIONS

- **Questions do not need to be rewritten** on the assignment sheet
- **Use the template and format** that have been provided
- **Plagiarism is strictly prohibited**
- Reports must be submitted in **PDF format** with the following naming convention: **CS_AslabCode_FullName_NPM_OS[No.Module]**
- **Grade reduction penalties** apply to anyone who commits a violation
- It is mandatory to cite references in **IEEE format**
- References must originate from **credible sources**

Copy all code, and screenshot all result or output

PART 1: Provider-Caller Architecture & Symbol Exporting

In this section, you are required to implement the inter-module communication architecture (*Provider-Caller*)

1. Create a file named math_provider.c. This module will serve as the *Provider*.
2. Within math_provider.c, implement two fundamental mathematical functions:
   a. calculate_area(int length, int width): Returns the area of a rectangle.
   b. get_secret_key(void): Returns a constant integer (e.g., 42).
3. Utilize the EXPORT_SYMBOL() macro to explicitly export the calculate_area function.
4. Utilize the EXPORT_SYMBOL_GPL() macro to explicitly export the get_secret_key function.
5. Create a secondary file named math_caller.c. This module will operate as the *Caller*.
6. During the initialization phase (init), math_caller.c must invoke both functions from the *Provider* and print the calculated results into the Kernel Ring Buffer.

PART 2: Multiple Independent Modules Compilation

Utilize the Kbuild build system mechanism to concurrently compile both source files into two distinct kernel modules, without isolating them into separate directories.

1. Construct a Makefile utilizing the obj-m variable such that a single make instruction generates two separate object files: math_provider.ko and math_caller.ko simultaneously.
2. Present the comprehensive contents of the Makefile you have constructed. **(Screenshot!)**

PART 3: Module Dependency & Load/Unload Rules

This section demonstrates how the kernel tracks inter-module dependencies and enforces runtime memory protection mechanisms.

1. Perform the module loading sequence utilizing insmod in the correct hierarchical order (the *Provider* first, followed by the *Caller*).

2. Inspect the *Kernel Symbol Table* by executing cat /proc/kallsyms | grep "calculate_area". Demonstrate that the symbol is successfully exported and registered within system memory. **(Screenshot!)**
3. Display the output of the lsmod | grep math_provider command and elaborate on the numerical value and significance of the Used by column. **(Screenshot!)**
4. Attempt to unload the math_provider module utilizing rmmod **while** the math_caller module remains actively operating in memory. Display the resulting terminal error proving that the system proactively rejects this execution. **(Screenshot!)**

PART 4: License Restrictions & Tainted Kernel

This section evaluates your comprehension of the stringent Open Source regulatory frameworks within Linux and their subsequent impact on EXPORT_SYMBOL_GPL.

1. Modify the math_caller.c file. Alter its license declaration from MODULE_LICENSE("GPL") to a closed-source license, specifically MODULE_LICENSE("Proprietary").
2. Re-execute the compilation process (make).
3. Attempt to reload the modified math_caller.ko module.
4. Inspect the Kernel Ring Buffer using the dmesg command. Exhibit the critical error or severe warning message generated by the kernel, explicitly stating the refusal to permit a *non-GPL* module to utilize a *GPL symbol*. **(Screenshot!)**
5. Verify the *Tainted Kernel* status by executing cat /proc/sys/kernel/tainted. The resultant value will no longer be 0. Briefly explain the systemic ramifications of a *tainted kernel* within a Linux environment! **(Screenshot!)**